A. 1) $N * (\log_2 N)^2 + N^2 * (\log_2 N)$

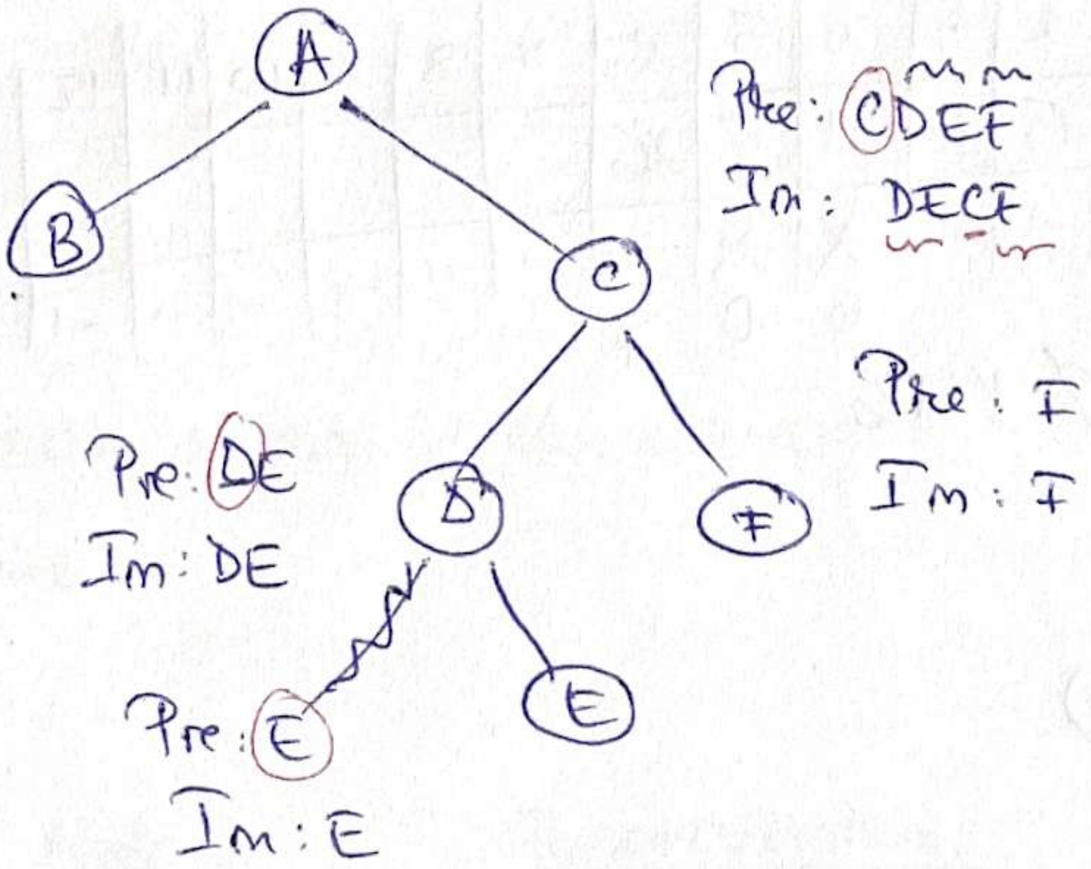
[a)]

[c)]

[d)]

2) Inorder: B A D E C F

Preorder: A B C D E F



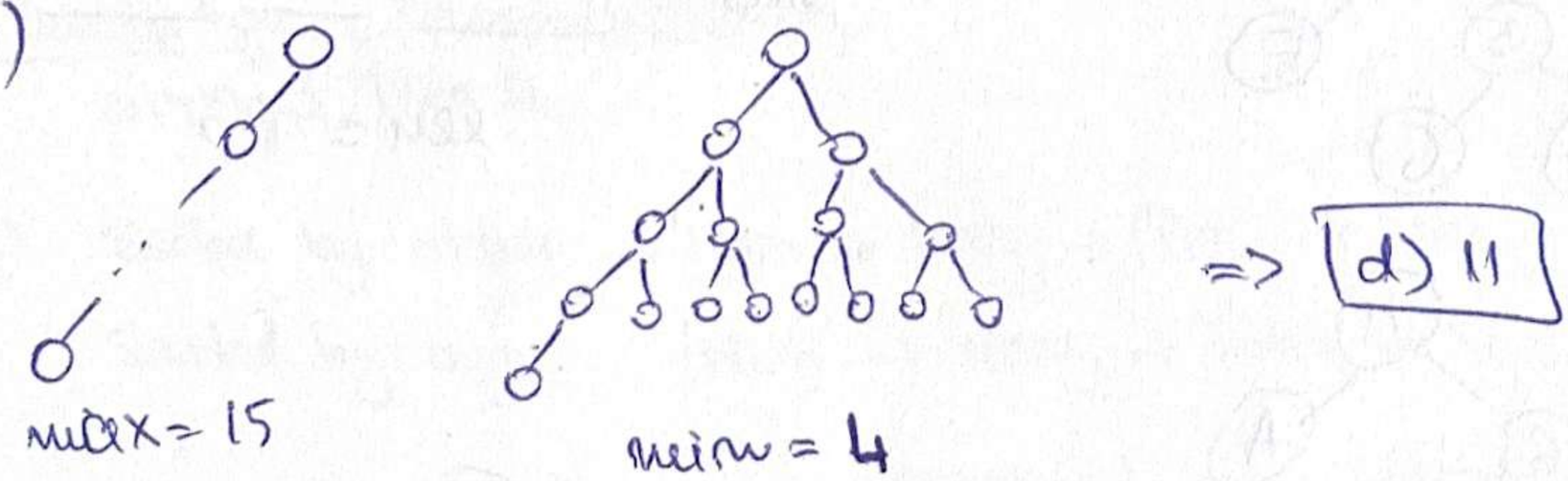
internal nodes: A, C, D => [a)], [c)], [d)]
(have children)

ROOT is also internal!

3) HT, 50 pos, collis. res. w open addr.

=> 1 elem/position => max [b)] 50 nr. can be inserted.

4)



=> [d) 11]

B. 2.1) same as row 3.

Lecture ex:

$\alpha = \frac{n}{m}$ = load factor ← stored, ↖ positions

$m = 13$; insert: 5, 18, 16, 15, 13, 31, 26.

Key	5	18	16	15	13	31	26
hash	5	5	3	2	0	5	0
	✓	✓	✓	✓	✓	✓	

31 → fE=4

26 → fE=6

13 → fE=1

18 => fE=0 => add there!

Pos	0	1	2	3	4	5	6	7	8	9	10	11	12
T	18	13	15	16	31	5	26						
next (empty)	~~X~~ 1	~~X~~ 4	-1	-1	~~X~~ 6	~~X~~ 0	~~X~~ 7	-1	-1	-1	-1	-1	-1

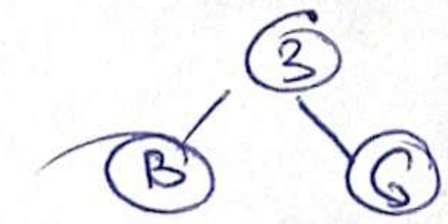
firstempty: ~~0~~ ~~X~~ ~~4~~ ~~X~~ 6

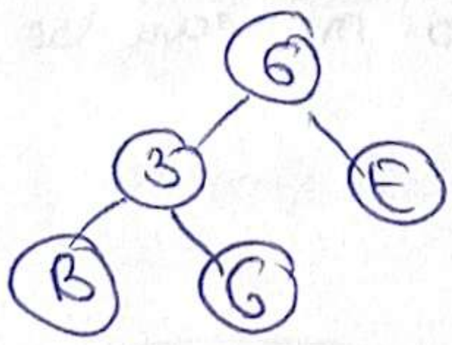
...

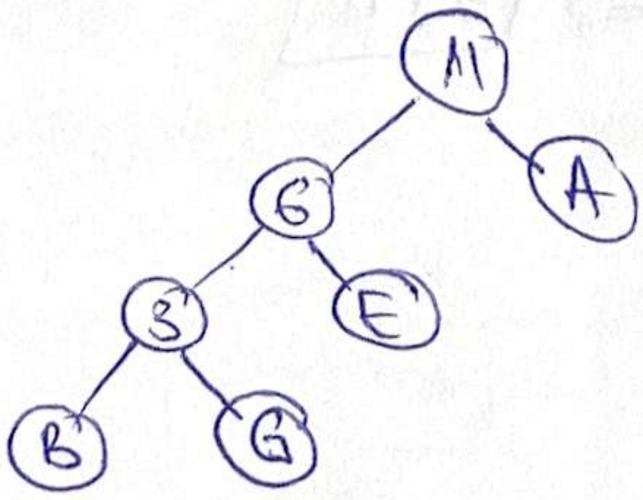
2.2) same as row 3)

2.3) Huffman.

pr.queue: (A) 5, (B) 2, (C) 8, (D) 7, (E) 3, (F) 7, (G) 1

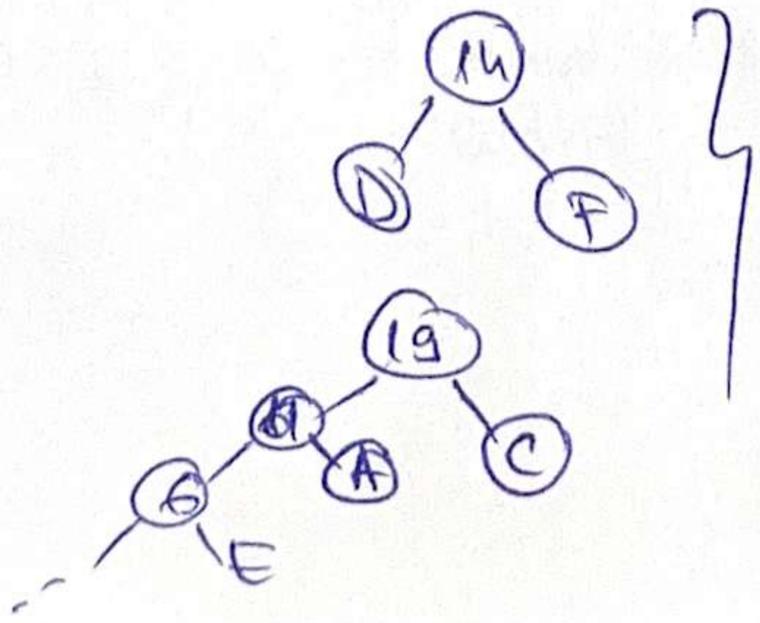




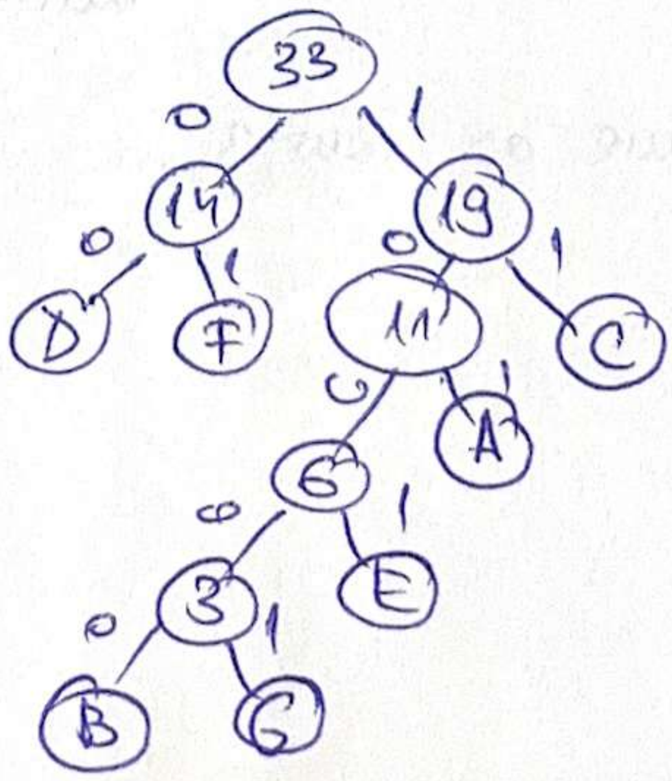


BAG: 100001010100001

len = 13



=>



3) (Peter, 19), (Ken, 16), (Sue, 21), (Mary, 18)

- Dyn. alloc. LL.
  - IteratorName => return ascending by name
  - IteratorAge => return ascending by age

  $\Theta(1)$ iterator operations.

a) repres. container + iterator

SLL:

SLLNode:
info: TElem
next: ↑Node

SLL:
head: TSLLNode

⊕ sorted => relation is part of the structure!
↳ by name
↳ by age

TPair:
name: TString
age: Integer

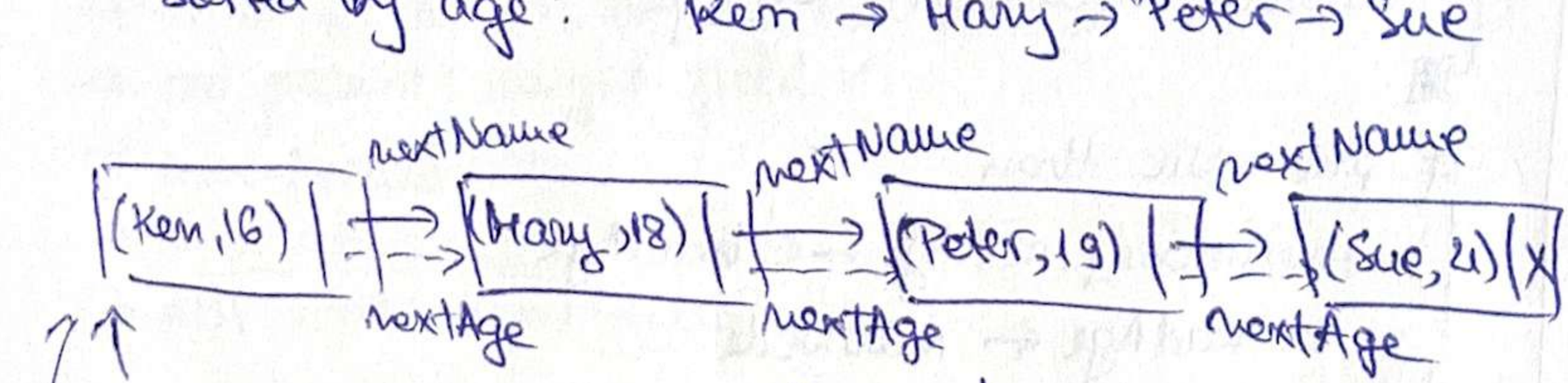
Node:
info: TPair
nextName TNode
nextAge: TNode

PersonCollection:
headName: ↑Node
headAge: ↑Node
relationName: Relation
relationAge: Relation
~~size: Integer~~

IteratorName: c: PersonCollection
current: ↑Node

IteratorAge: c: PersonCollection
current: ↑Node

b) Sorted by name: Ken → Mary → Peter → Sue
Sorted by age: Ken → Mary → Peter → Sue



=> same nodes, not copies, but each node has 2 links.

c) add → specify, implement, BC, WC, total.

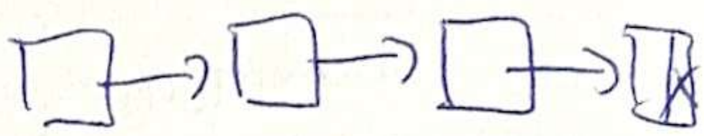
↳ allocate a new node, store elem init.

→ find pos. in name-sorted-list (relationName) → insert new node by updating nextName. → BC: $\Theta(1)$, WC = $\Theta(n)$; update: $\Theta(1)$

→ ——— age-sorted-list (relationAge) → ——— same node by updating nextAge. → BC: $\Theta(1)$, WC = $\Theta(n)$; update: $\Theta(1)$

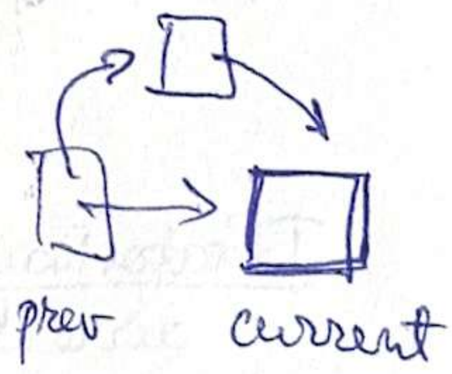
We cannot use the iterators because they don't provide prev and we need prev in order to insert in the list!

⇒ Total: $O(n)$



```
subalgorithm add (c, elem) is
    newNode ← allocate.
    [newNode].info ← elem  //store pair
    [newNode].nextName ← NIL ; [newNode].nextAge ← NIL
    prev ← NIL
    current ← c.headName
    while current ≠ NIL and c.relationName([current].info.name, elem.name)
        prev ← current
        current ← [current].nextName
    if prev = NIL then  <= it's empty, the newly added node is the only node. OR elem comes before the first Node
        [newNode].nextName ← c.headName
        c.headName ← newNode // head becomes the newNode.
    else <= insert between prev and current
        [newNode].nextName = current
        [prev].nextName = newNode
```

insert in name-sorted list




```
    prev ← NIL <= reset previous
    current ← c.headAge
    while current ≠ NIL and c.relationAge([current].info.age, elem.age)
        prev ← current
        current ← [current].nextAge
    if prev = NIL then
        [newNode].nextAge ← c.headAge
        c.headAge ← newNode
    else
        [newNode].nextAge = current
        [prev].nextAge = newNode
```

insert in age-sorted list

$(\log_2 N)^2 + N^2 * (\log_2 N)$

d) IteratorAge

first / init ?
getCurrent (it)
next (it)
valid (it)

IteratorAge
c: PersonCollection
current. ↑Node

subalgorithm initAge(it, c) is
    it.c ← c
    it.current ← c.headAge

function valid (it) is
    valid ← it.current ≠ NiL

if ≠ NiL
    valid ← True
else
    valid ← False

function getCurrent (it) is
    if not valid(it) then
        @throw...
    else
    getCurrent ← [it.current].info

subalgorithm next(it) is
    if not valid (it) then
        @throw....
    it.current ← [it.current].nextAge

Θ(1)

4) BT = ordered tree with max 2 children (we can have gaps => no heap str.)
↳ MiN binary-heap property => { parent ≤ left, parent ≤ right }

→ repres. of BT. (ll w dyn. alloc.) ✓
→ no parent / height field ✓
→ non-recursively
→ one traversal => Level-order traversal using a Queue.
→ ADT stack / Map / Queue etc.

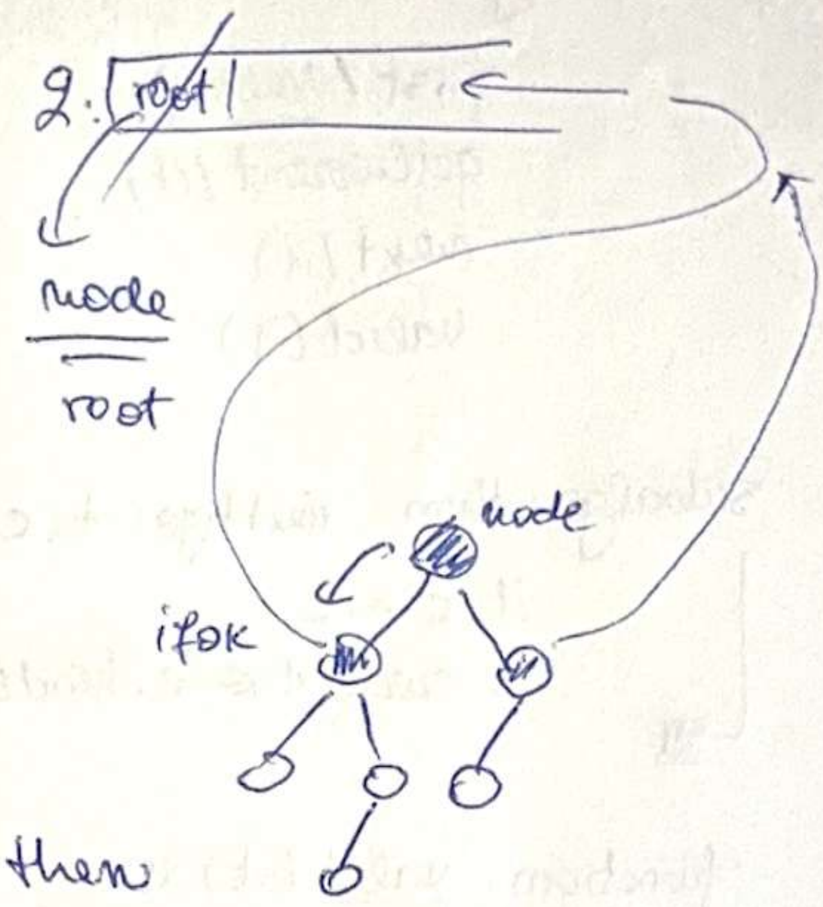
!!
Queue of ↑Node
↓
Operations:
- init(q)
- push(q, e)
- pop(q)
- top(q)
- isEmpty(q)

a) Node:
info: TElem
left: ↑Node
right: ↑Node

BT:
root: ↑Node

$\sqrt{2^N})^2 + N^2 * 11$

```
function isMinHeap(bt)
    if bt.root = Nil then
        return isMinHeap ← True
    init(q)
    push(q, bt.root)
    while not isEmpty(bt) ex
        node ← top(q)
        pop(q)
        if [node].left ≠ Nil then
            if [node].info > [[node].left].info then
                isMinHeap ← False
            push(q, [node].left)
        if [node].right ≠ Nil then
            if [node].info > [[node].right].info then
                isMinHeap ← False
            push(q, [node].right)
    isMinHeap ← True
```



→ we visit each node once, for every node we compare value with its children, if one comparison fails, return false

~~$\Theta(n)$~~ BC: $\Theta(1)$ → BT empty OR
→ root > one child.

WC: $\Theta(n)$ → BT ~~is a~~ has a min heap property OR
→ last node does not check the property

$T: O(n)$